PROJETO PRÁTICO · ACOMPANHAMENTO DO ESTUDANTE

Da ideia à solução com IA

Checklist de desenvolvimento com plataformas digitais e Vibecode / IA. Marque apenas o que já foi realizado e registre as evidências de cada etapa.

1. Definir o desafio → 2. Escolher a plataforma → 3. Integrar IA → 4. Testar e validar → 5. Documentar

Nomes dos estudantes (mínimo 2 e máximo 3)

Estudante 1 (obrigatório)

Brayan Aragão Pletsch

Estudante 2 (obrigatório)

Pedro Freire de Farias

Estudante 3 (opcional)

Davi José Nascimento

Turma / disciplina

Tendências em Ciência da Computação

Nome do projeto

Sistema de Cadastro e Automação de Usuários

Data do preenchimento

30/10/2026

01

08/10

Definir o desafio

Escolha um problema simples e com escopo bem definido.

- ☑ Descrevi o problema e o público que será beneficiado.
- ☑ Defini o objetivo e o resultado esperado.
- ☑ Delimitei o que a solução fará e o que ficará fora do escopo.
- ☑ Estabeleci critérios para verificar se o desafio foi resolvido.

Problema, público, objetivo e critérios de sucesso

O problema que escolhemos é o cadastro de pessoas feito de forma manual. Normalmente alguém anota os dados, depois passa tudo para uma planilha e ainda precisa mandar o e-mail de confirmação para cada pessoa. Isso toma tempo, aumenta a chance de erro de digitação e às vezes a pessoa demora para ter uma resposta.

O projeto ajuda quem recebe esses cadastros, como uma secretaria, um setor de atendimento ou a organização de um evento, e também quem se cadastra.

Nosso objetivo é fazer um protótipo em que a pessoa preenche um formulário online, os dados vão direto para uma planilha e ela recebe um e-mail confirmando o cadastro, sem ninguém precisar fazer isso na mão.

A solução vai ter o formulário com nome e e-mail, o salvamento automático na planilha, a conferência do e-mail digitado e o envio da confirmação. Não vamos fazer login com senha, painel de administração, aplicativo de celular, pagamento nem banco de dados próprio.

Vamos considerar que deu certo se, em 10 cadastros de teste, todos aparecerem na planilha sem perder nem repetir nenhum, todo cadastro válido receber o e-mail, um cadastro com e-mail inválido não for aceito e o processo funcionar do começo ao fim sem ninguém precisar intervir.

02

15/10

Escolher e configurar a plataforma

Selecione a ferramenta adequada ao caso de uso: Bubble, Make, Zapier, Airtable, Pipefy, Adalo ou outra.

- ☑ Comparei as ferramentas considerando o caso de uso.
- ☑ Verifiquei limites de uso, custos e recursos necessários.
- ☑ Escolhi a plataforma e justifiquei minha decisão.
- ☑ Configurei o ambiente e executei um teste inicial.

Plataforma escolhida, justificativa e configuração realizada

Escolhemos o Make, no plano gratuito, junto com o Google Forms e o Google Sheets.

Antes de decidir, comparamos Make, Zapier, Airtable, Pipefy, Bubble e Adalo. Bubble e Adalo servem para criar aplicativos completos, e o nosso desafio é bem mais simples que isso. O Pipefy é mais voltado para processos com etapas de aprovação. O Airtable seria bom para guardar os dados, mas sozinho não manda o e-mail. O Zapier chegou perto, só que no plano grátis cada automação pode ter só 2 passos e são apenas 100 tarefas por mês. Já o Make grátis tem 1.000 créditos por mês e deixa montar o fluxo inteiro com vários passos, então atende bem o protótipo.

Também olhamos os limites do plano gratuito do Make: são só 2 cenários ativos, o intervalo mínimo entre execuções é de 15 minutos e cada verificação gasta crédito. Por isso, durante os testes, vamos rodar o cenário manualmente.

Na configuração, criamos o formulário no Google Forms ligado a uma planilha e montamos no Make um cenário com o módulo Watch New Rows, que lê as linhas novas da planilha. Fizemos um teste com dados inventados e o Make pegou o cadastro corretamente.

03

22/10

Integrar Vibecode / IA

Dialogue com a IA de forma iterativa para estruturar configurações, lógicas leves ou integrações.

- ☑ Escrevi um prompt com objetivo, contexto e restrições.
- ☑ Revisei a resposta da IA e solicitei ajustes quando necessário.
- ☑ Implementei uma lógica, configuração ou integração compatível com o desafio.
- ☑ Registrei os prompts, as alterações e os resultados obtidos.
- ☑ Mantive senhas, chaves de API e dados pessoais fora dos prompts e do código público.

Prompts utilizados, integração implementada e ajustes realizados

Usamos o Claude (Anthropic) para montar a lógica do cenário no Make. No primeiro prompt passamos o problema, o objetivo, os limites do plano gratuito do Make e as restrições: usar só módulos do plano grátis, não mandar nenhum dado de cadastro para a IA e não inventar nome de módulo, campo ou função.

A IA propôs um Router com rotas para cadastro válido, repetido e inválido, uma coluna de status na planilha, duas fórmulas (uma que padroniza o e-mail e outra que conta se ele já apareceu antes), uma expressão regular para conferir o e-mail e o texto da confirmação.

Antes de aceitar, conferimos a resposta na documentação do Make e do Google e testamos a expressão regular no Node. Três pontos não batiam com a documentação atual (a conexão do Gmail, o nome do tratamento de erro e a validação do Forms) e achamos duas brechas na expressão. Também vimos que agendar a cada 15 minutos gastaria mais que os 1.000 créditos do mês, então o agendamento passou para 60 minutos. Mandamos essas correções num segundo prompt, e a versão final passou em todos os casos que testamos.

Com esse desenho montamos o formulário, a planilha e o cenário no Make. Na montagem ainda ajustamos duas coisas: a aba de respostas veio como tabela e não aceitava fórmula no cabeçalho, e a data só saiu no formato certo depois que colocamos o formato entre aspas.

Os prompts, as respostas da IA sem edição e o que mudamos estão no GitHub. Nenhum dado de cadastro, senha ou chave foi para os prompts, e as conexões com o Google ficaram guardadas no próprio Make.

04

29/10

Testar e verificar a segurança

Teste toda lógica gerada por IA. Valide entradas e trate cenários de erro.

- ☑ Testei o fluxo principal com dados fictícios.
- ☑ Testei campos vazios, dados inválidos e entradas inesperadas.
- ☑ Verifiquei mensagens de erro e falhas de integração.
- ☑ Conferi permissões de acesso e proteção de credenciais.
- ☑ Corrigi problemas e repeti os testes.
- ☑ Comparei os resultados com os critérios de sucesso.

Casos de teste, resultados, falhas e correções

Testamos com dados fictícios e caixas de e-mail temporárias, seguindo uma lista de casos que montamos a partir dos critérios da etapa 1.

No fluxo principal, os cadastros válidos foram gravados na planilha com status ENVIADO e o e-mail de confirmação chegou só com o primeiro nome da pessoa, inclusive com acento ("Olá, José!") e num e-mail institucional com subdomínio. Um e-mail digitado com letras maiúsculas e espaços, como acontece no celular, foi padronizado e também recebeu a confirmação. O mesmo e-mail cadastrado de novo e a mesma pessoa mandando o formulário duas vezes seguidas ficaram como DUPLICADO, sem segundo e-mail. Um nome com código HTML apareceu no e-mail como texto, sem mudar a formatação.

Nas entradas inválidas, o formulário recusou e-mail sem @, sem terminação e começando com ponto, e também o envio com o nome em branco. A mensagem de erro aparece na tela, e mesmo mandando direto para o servidor do formulário a resposta foi recusada e não chegou na planilha. Uma execução do Make sem cadastro novo não processou nenhuma linha.

Na primeira execução achamos uma falha na integração entre o Make e a planilha: a data de processamento foi gravada no formato ISO. Corrigimos o formato no Make, repetimos o teste e a data saiu certa. O cenário também tem uma rota de erro para quando o Gmail falhar, que grava ERRO_ENVIO na planilha, mas não chegamos a forçar uma falha de envio.

A planilha está com acesso restrito, o formulário não pede login nem coleta o e-mail da conta Google, e o repositório não tem link do formulário, senha, token nem os e-mails usados nos testes. Nos prints, os endereços foram borrados.

Comparando com os critérios de sucesso: rodamos 8 dos 11 envios planejados e todos apareceram na planilha sem repetir, todos os cadastros válidos receberam o e-mail e o formulário barrou os e-mails inválidos. Ficaram sem teste a conferência do e-mail inválido dentro do Make e o

agendamento automático, então o critério de funcionar sem ninguém intervir ainda não foi comprovado. Os casos e resultados estão no casos-de-teste.md do GitHub.

05

Consolidação final

Documentar a solução

5 / 5

Consolide os registros do projeto e as instruções para utilização. Prazo conforme orientação docente.

- ☑ Descrevi o problema, a solução e a plataforma utilizada.
- ☑ Registrei como configurar e utilizar a solução.
- ☑ Incluí evidências dos testes e das melhorias.
- ☑ Registrei limitações, referências e uso de IA.
- ☑ Revisei os registros antes de imprimir ou salvar em PDF.

Link do projeto, evidências, limitações e próximos passos

A documentação completa está no GitHub, na pasta unidade-3-low-code/projeto-cadastro-automacao do repositório https://github.com/BrayanPletsch/Tendencias_CienciaComputacao_2026_2.

Lá estão a descrição do desafio, a justificativa da escolha do Make, o desenho do fluxo, o guia de montagem campo a campo, os prompts com as respostas da IA, os casos de teste com os resultados, os prints dos testes (com os e-mails borrados) e os slides da apresentação. O uso de IA e as fontes consultadas estão no README do projeto.

As principais limitações são o plano gratuito do Make, que obriga a checar a planilha só de hora em hora para não passar dos créditos, a dependência de ninguém apagar, ordenar ou digitar em cima das fórmulas da planilha e o fato de a expressão regular conferir o formato do e-mail, mas não se ele existe.

Como próximos passos, queremos rodar os testes que ficaram de fora (falha do Gmail, e-mail inválido chegando ao Make e agendamento automático), confirmar o endereço por um link no e-mail, trocar a checagem periódica por um gatilho instantâneo e colocar um aviso de privacidade no formulário.

```
```